

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»
Факультет прикладной информатики

Мобильная разработка
Отчет по курсовой работе на тему:
Разработка мобильного приложения с использованием Kotlin

Выполнил:
студент группы К3444
Федоров Л. А.

Проверил:
Преподаватель ФПИН ИТМО
Шатинский Г. С.

Санкт-Петербург

2026 г.

Содержание

1. Введение
2. Лабораторная работа №1: Activity, Fragment и Navigation
3. Лабораторная работа №2: MVVM, ViewModel и LiveData
4. Лабораторная работа №3: API, Room и Repository
5. Лабораторная работа №4: UI, WorkManager и Notifications
6. Заключение

1. Введение

В рамках данной курсовой работы было разработано мобильное Android-приложение "MIN" — мессенджер с базовым функционалом для демонстрации ключевых концепций и технологий разработки под Android.

Цель работы:

Разработать Android-приложение с использованием современных архитектурных подходов и компонентов Android Jetpack.

Задачи:

- Реализовать навигацию с использованием Navigation Component
- Применить архитектуру MVVM для разделения логики и представления
- Интегрировать работу с сетью через Retrofit и локальное хранение через Room
- Реализовать фоновую синхронизацию с помощью WorkManager
- Применить компоненты Material Design для современного UI

Используемые технологии:

- Kotlin — основной язык разработки
- Android Jetpack (Navigation, ViewModel, LiveData, Room, WorkManager)
- Retrofit + OkHttp — работа с сетью
- Coil — загрузка изображений
- Material Design Components — UI компоненты
- Coroutines + Flow — асинхронное программирование

2. Лабораторная работа №1

Activity, Fragment и Navigation

Задача:

Разработать Android-приложение "Мессенджер" с использованием архитектуры на основе Activity и Fragment, а также встроенной навигации.

Функциональные требования:

1. Приложение содержит одну главную активность (MainActivity) — точку входа
2. Реализована навигация с помощью Bottom Navigation
3. Три экрана: Лента, Профиль, Настройки
4. Навигация через NavController и Navigation Graph
5. Отслеживание жизненного цикла с выводом в лог

Реализация:

MainActivity.kt — главная активность приложения:

```
class MainActivity : AppCompatActivity() {  
    private lateinit var binding: ActivityMainBinding  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        Log.d("MainActivity", "onCreate")  
  
        binding = ActivityMainBinding.inflate(layoutInflater)  
        setContentView(binding.root)  
  
        val navHostFragment = supportFragmentManager  
            .findFragmentById(R.id.nav_host_fragment) as NavHostFragment  
        val navController = navHostFragment.navController  
  
        binding.bottomNavigation.setupWithNavController(navController)  
    }  
}
```

```

override fun onStart() {
    super.onStart()
    Log.d("MainActivity", "onStart")
}
// ... остальные методы жизненного цикла
}

```

nav_graph.xml — граф навигации:

```

<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/nav_graph"
    app:startDestination="@id/feedFragment">

    <fragment
        android:id="@+id/feedFragment"
        android:name="com.example.mymessenger.FeedFragment"
        android:label="@string/feed" />

    <fragment
        android:id="@+id/profileFragment"
        android:name="com.example.mymessenger.ProfileFragment"
        android:label="@string/profile" />

    <fragment
        android:id="@+id/settingsFragment"
        android:name="com.example.mymessenger.SettingsFragment"
        android:label="@string/settings" />
</navigation>

```



Рисунок 1 – Навигация приложения + экран Ленты

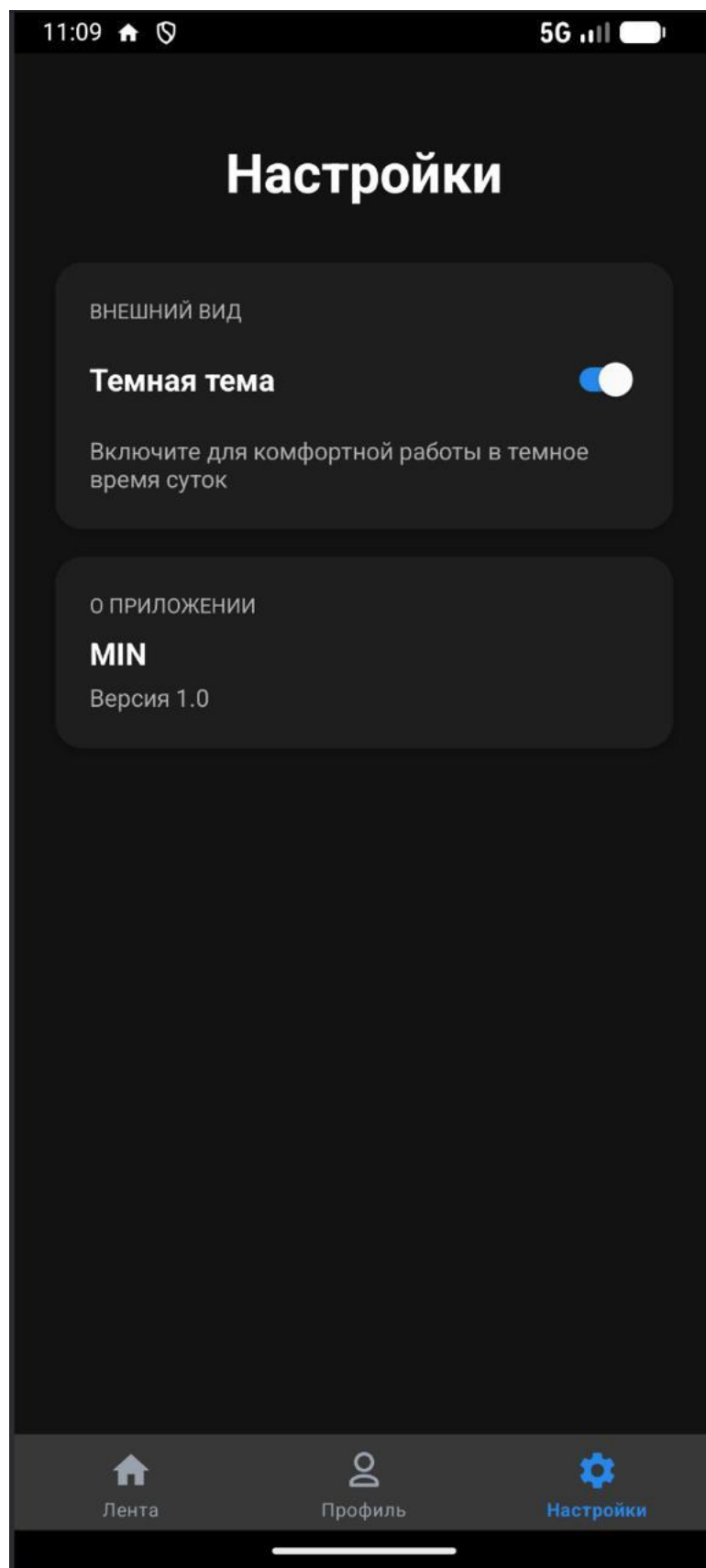


Рисунок 2 – экран Настроек

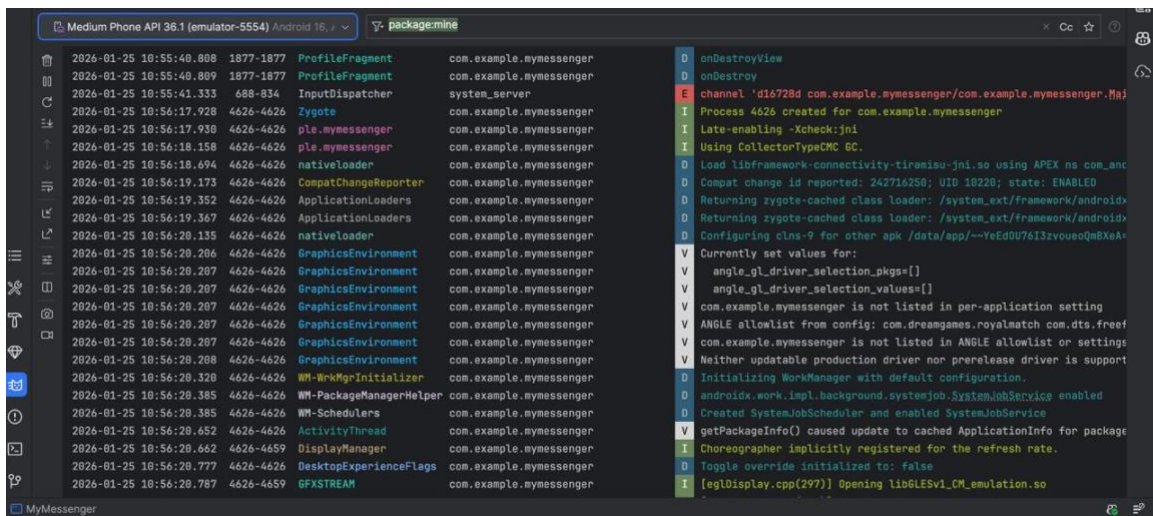


Рисунок 3 – Полное логирование через LogCat

3. Лабораторная работа №2

MVVM, ViewModel и LiveData

Задача:

Реализовать управление состоянием приложения и архитектуру MVVM для хранения и обновления пользовательских данных.

Функциональные требования:

1. ViewModel для хранения состояния профиля и настроек
2. Экран "Профиль" с редактируемым именем и статусом
3. Экран "Настройки" с переключателем темы (светлая/тёмная)
4. Сохранение данных при повороте экрана
5. Логирование жизненного цикла ViewModel

Реализация:

ProfileViewModel.kt:

```
class ProfileViewModel(application: Application) : AndroidViewModel(application)
{
    private val preferencesManager = PreferencesManager(application)

    private val _userName = MutableLiveData<String>().apply {
        value = preferencesManager.userName
    }
    val userName: LiveData<String> = _userName

    private val _userStatus = MutableLiveData<String>().apply {
        value = preferencesManager.userStatus
    }
    val userStatus: LiveData<String> = _userStatus

    init {
        Log.d("ProfileViewModel", "ViewModel created")
    }
}
```

```

    }

    fun updateUserName(name: String) {
        Log.d("ProfileViewModel", "updateUserName: $name")
        preferencesManager.userName = name
        _userName.value = name
    }

    override fun onCleared() {
        super.onCleared()
        Log.d("ProfileViewModel", "ViewModel destroyed")
    }
}

```

SettingsViewModel.kt:

```

class SettingsViewModel(application: Application) : AndroidViewModel(application)
{
    private val preferencesManager = PreferencesManager(application)

    private val _isDarkTheme = MutableLiveData<Boolean>().apply {
        value = preferencesManager.isDarkTheme
    }
    val isDarkTheme: LiveData<Boolean> = _isDarkTheme

    fun setDarkTheme(isDark: Boolean) {
        Log.d("SettingsViewModel", "setDarkTheme: $isDark")
        preferencesManager.isDarkTheme = isDark
        _isDarkTheme.value = isDark
    }
}

```

PreferencesManager.kt — сохранение данных в SharedPreferences:

```
class PreferencesManager(context: Context) {  
    private val prefs: SharedPreferences = context.getSharedPreferences(  
        PREFS_NAME, Context.MODE_PRIVATE  
    )  
  
    var userName: String  
        get() = prefs.getString(KEY_USER_NAME, DEFAULT_NAME) ?:  
        DEFAULT_NAME  
        set(value) = prefs.edit().putString(KEY_USER_NAME, value).apply()  
  
    var isDarkTheme: Boolean  
        get() = prefs.getBoolean(KEY_DARK_THEME, false)  
        set(value) = prefs.edit().putBoolean(KEY_DARK_THEME, value).apply()  
}
```

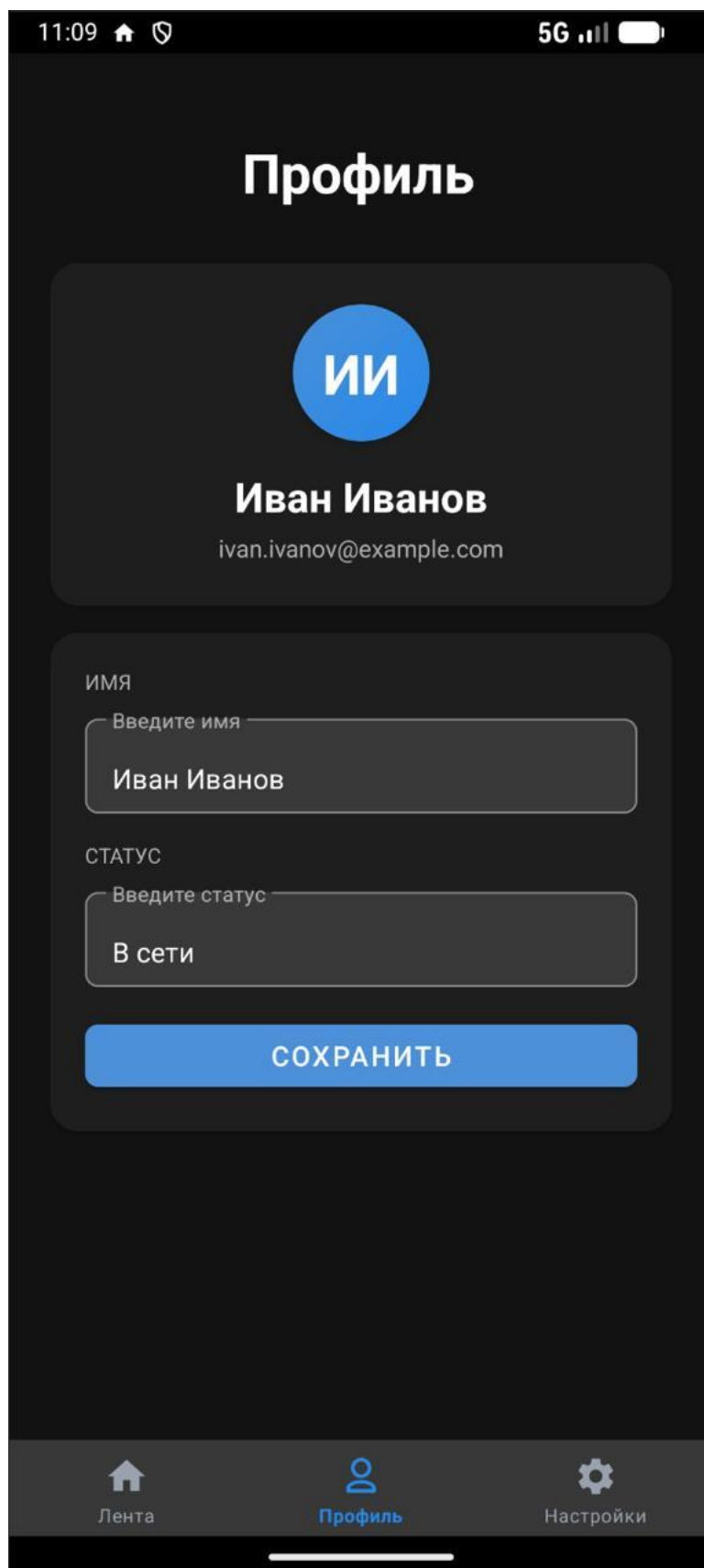


Рисунок 4 – Страница профиля

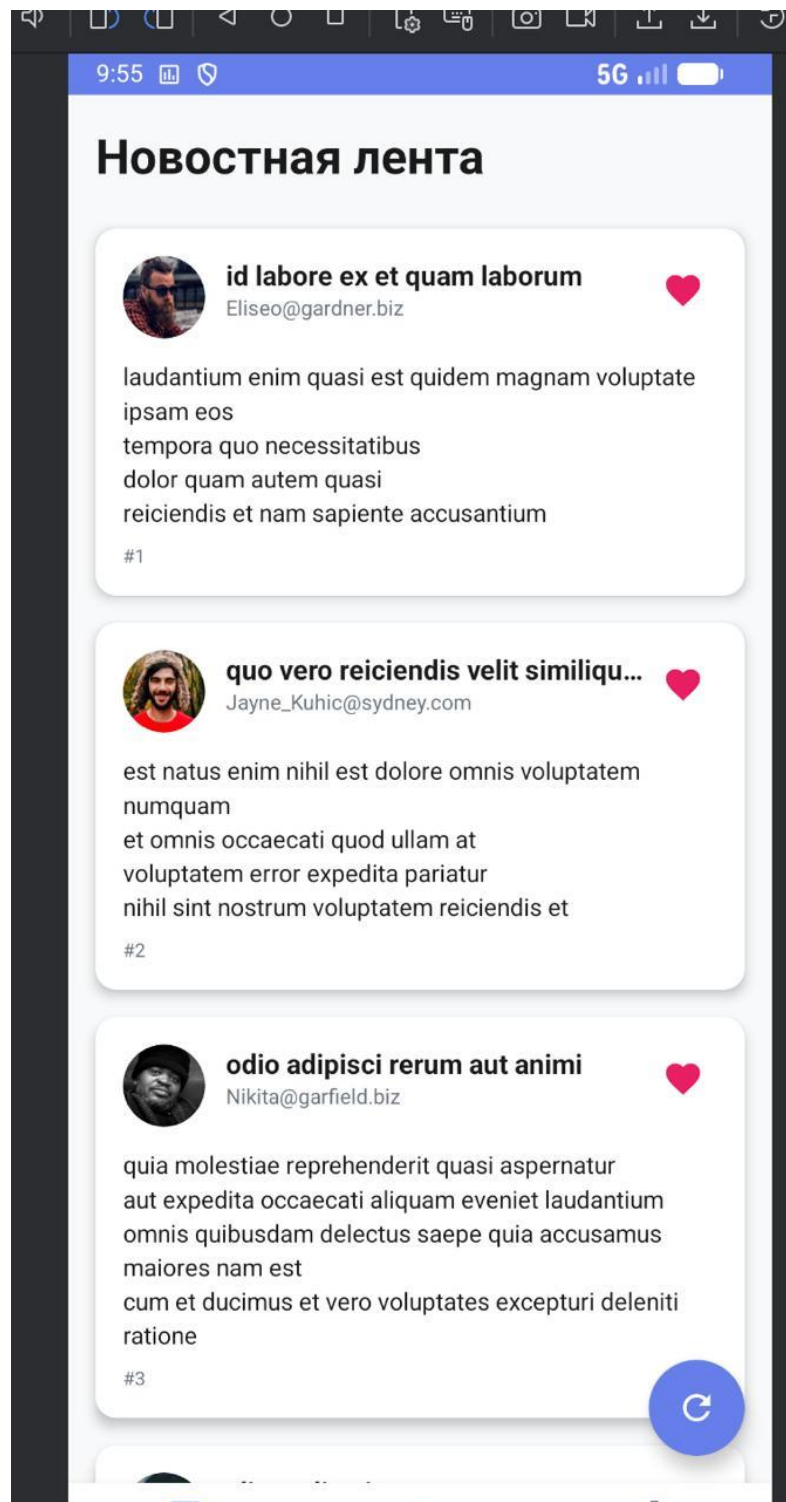


Рисунок 5 – Страница ленты в белой теме

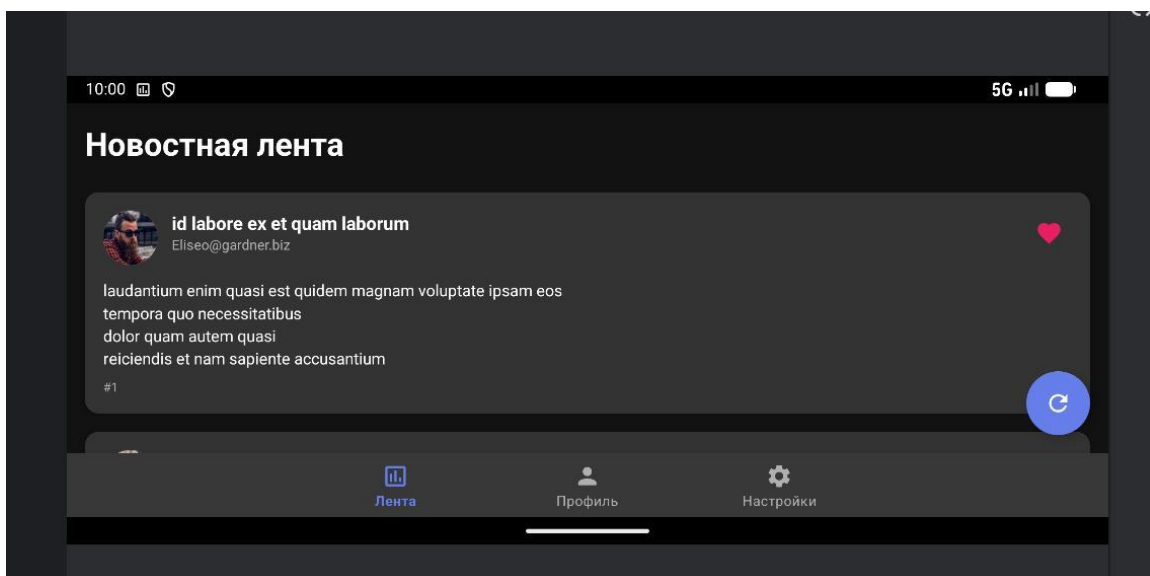


Рисунок 6 – Страница ленты в темной теме в перевернутом на 90 градусов экране

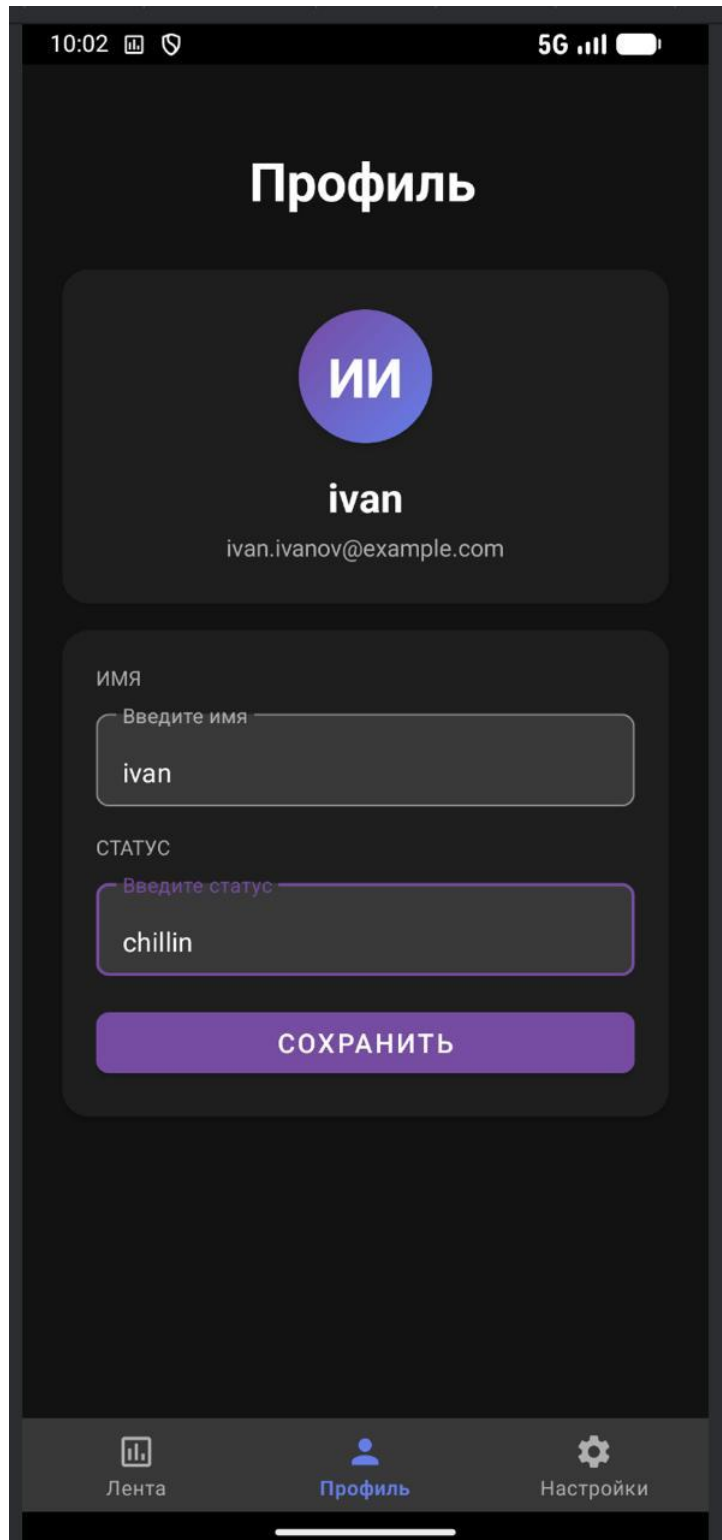


Рисунок 7 – экран профиля с установленными значениями в полях имени и статуса

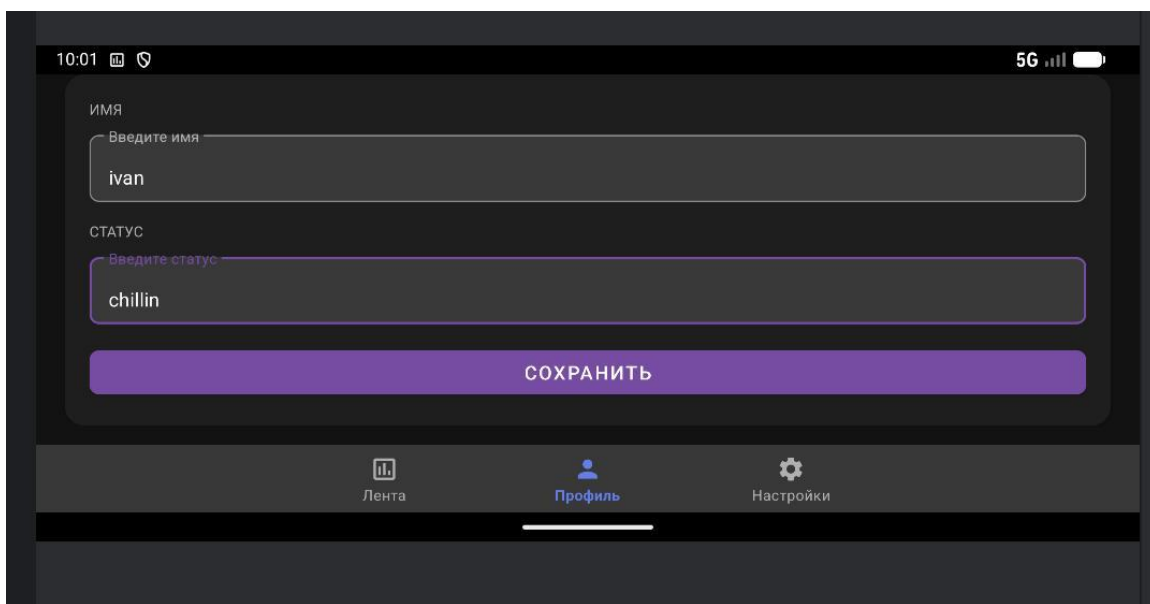


Рисунок 8 – перевернутый экран профиля с неизменившимися установленными до этого значениями в полях имени и статуса

4. Лабораторная работа №3

API, Room и Repository

Задача:

Добавить загрузку списка сообщений из API и реализовать сохранение данных в локальную базу Room для офлайн-доступа.

Функциональные требования:

1. MessageRepository для получения данных из API
2. Retrofit-клиент с корутинами
3. Отображение списка сообщений в RecyclerView
4. Сохранение данных в Room
5. Загрузка из кэша при отсутствии сети
6. Кнопка "Обновить" для ручного обновления

Реализация:

Message.kt — Entity для Room:

```
@Entity(tableName = "messages")
data class Message(
    @PrimaryKey
    @SerializedName("id")
    val id: Int,

    @SerializedName("postId")
    val postId: Int,

    @SerializedName("name")
    val name: String?,

    @SerializedName("email")
    val email: String?,
```

```

@SerializedName("body")
val body: String?,

val timestamp: Long = System.currentTimeMillis(),
val isLiked: Boolean = false
) {
    fun getAvatarUrlGenerated(): String {
        return "https://i.pravatar.cc/150?u=\${email ?: "default"}"
    }
}

```

ApiService.kt — интерфейс Retrofit:

```

interface ApiService {
    @GET("comments")
    suspend fun getMessages(
        @Query("_limit") limit: Int = 50
    ): List<Message>
}

```

MessageRepository.kt:

```

class MessageRepository(
    private val messageDao: MessageDao,
    private val apiService: ApiService,
    private val context: Context
) {
    fun getMessagesFromDatabase(): Flow<List<Message>> {
        return messageDao.getAllMessages()
    }

    suspend fun refreshMessages(): Resource<List<Message>> {
        return withContext(Dispatchers.IO) {
            try {

```

```

        if (!NetworkUtils.isNetworkAvailable(context)) {
            val cachedMessages = messageDao.getAllMessagesOnce()
            return@withContext Resource.Error(
                "Нет подключения. Показаны данные из кэша.",
                cachedMessages
            )
        }

        val messages = apiService.getMessages(limit = 50)

        // Сохраняем статус лайков
        val existingMessages = messageDao.getAllMessagesOnce()
        val likedIds = existingMessages.filter { it.isLiked }
            .map { it.id }.toSet()

        val messagesWithLikes = messages.map { message ->
            if (likedIds.contains(message.id)) {
                message.copy(isLiked = true)
            } else message
        }

        messageDao.insertMessages(messagesWithLikes)
        Resource.Success(messages)
    } catch (e: Exception) {
        val cached = messageDao.getAllMessagesOnce()
        Resource.Error("Ошибка: \${e.message}", cached)
    }
}
}
}
}

```

MessageDao.kt — DAO для работы с базой:

```

@Dao
interface MessageDao {
    @Query("SELECT * FROM messages ORDER BY timestamp DESC")

```

```
fun getAllMessages(): Flow<List<Message>>

@Insert(onConflict = OnConflictStrategy.REPLACE)
suspend fun insertMessages(messages: List<Message>)

@Query("UPDATE messages SET isLiked = :isLiked WHERE id = :messageId")
suspend fun updateLikeStatus(messageId: Int, isLiked: Boolean)
}
```

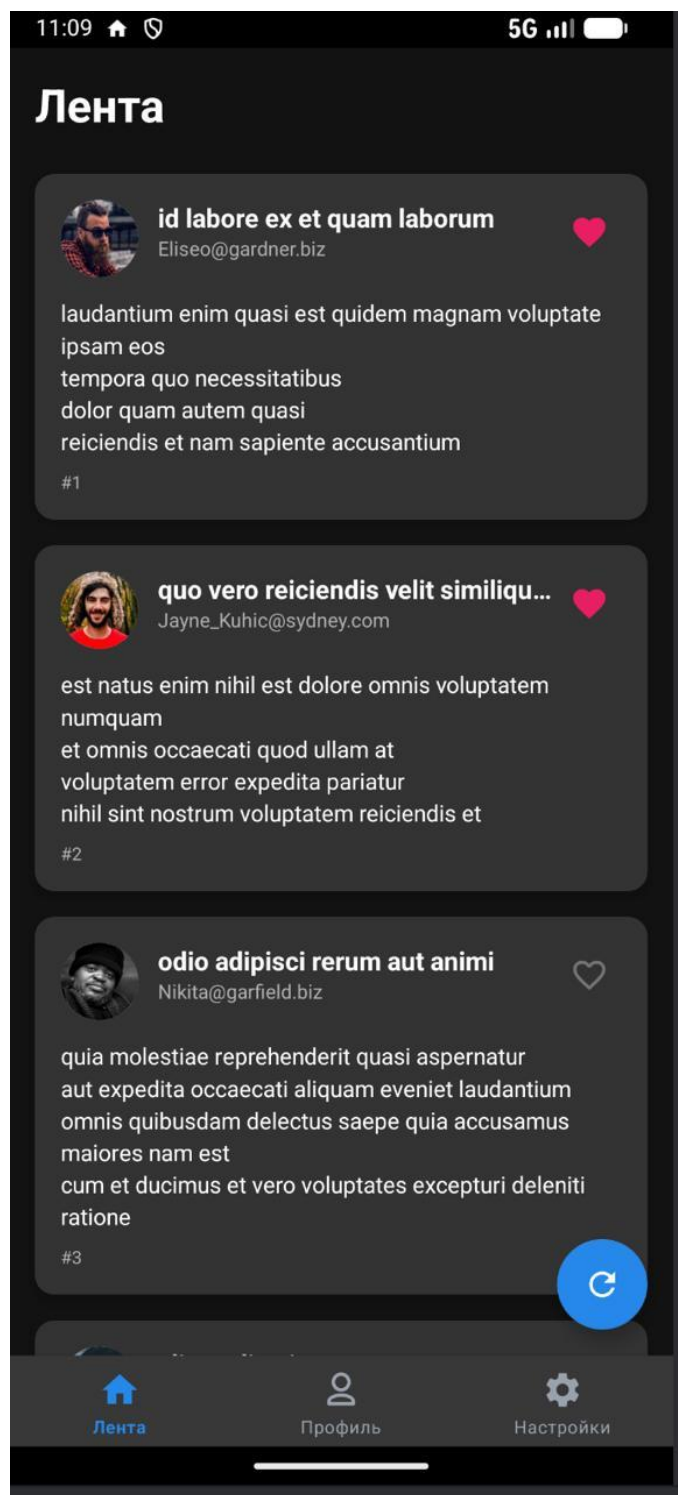


Рисунок 9 – страничка ленты с реализованной кнопкой обновления ленты

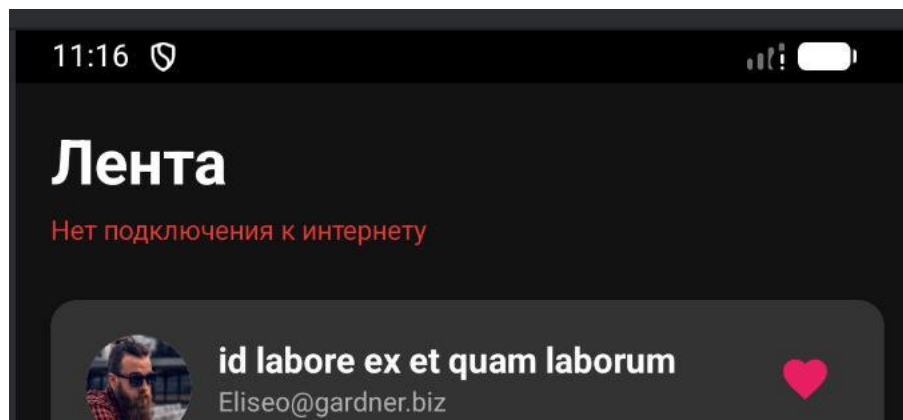


Рисунок 10 – страничка ленты с реализованной подгрузкой данных из кэша в отсутствии интернета и также вывод информации о том что нет интернета

5. Лабораторная работа №4

UI, WorkManager и Notifications

Задача:

Улучшить пользовательский интерфейс, добавить уведомления и фоновую синхронизацию сообщений.

Функциональные требования:

1. Кастомный RecyclerView.Adapter с аватаром, именем, текстом
2. Material Design: CardView, FloatingActionButton, AppBarLayout
3. Лайки сообщений с изменением иконки
4. WorkManager для периодической синхронизации
5. Notification при успешной синхронизации
6. Запрос разрешений при запуске
7. Отслеживание состояния сети (онлайн/офлайн)

Примеры реализации из проекта:

MessageAdapter.kt — кастомный адаптер:

```
class MessageAdapter(  
    private val onLikeClick: (Message) -> Unit  
) : ListAdapter<Message,  
    MessageAdapter.MessageViewHolder>(MessageDiffCallback()) {  
  
    class MessageViewHolder(  
        private val binding: ItemMessageBinding,  
        private val onLikeClick: (Message) -> Unit  
    ) : RecyclerView.ViewHolder(binding.root) {  
  
        fun bind(message: Message) {
```

```

binding.apply {
    textMessageName.text = message.name
    textMessageEmail.text = message.email
    textMessageBody.text = message.body

    imageAvatar.load(message.getAvatarUrlGenerated()) {
        crossfade(true)
        transformations(CircleCropTransformation())
    }

    updateLikeButton(message.isLiked)

    buttonLike.setOnClickListener {
        onLikeClick(message)
    }
}

private fun updateLikeButton(isLiked: Boolean) {
    binding.buttonLike.setImageResource(
        if (isLiked) R.drawable.ic_heart_filled
        else R.drawable.ic_heart_outline
    )
}
}
}

```

SyncWorker.kt — фоновая синхронизация:

```

class SyncWorker(
    context: Context,
    workerParams: WorkerParameters
) : CoroutineWorker(context, workerParams) {

    companion object {
        const val WORK_NAME = "sync_messages_work"
    }
}

```



```

    }

    override suspend fun doWork(): Result {
        Log.d("SyncWorker", "Starting sync...")

        return try {
            val database = AppDatabase.getDatabase(applicationContext)
            val repository = MessageRepository(
                database.messageDao(),
                RetrofitClient.apiService,
                applicationContext
            )

            when (val result = repository.refreshMessages()) {
                is Resource.Success -> {
                    NotificationHelper.showSyncSuccessNotification(
                        applicationContext
                    )
                    Result.success()
                }
                is Resource.Error -> Result.retry()
                else -> Result.success()
            }
        } catch (e: Exception) {
            Result.retry()
        }
    }
}

```

Настройка WorkManager в MainActivity:

```

private fun setupPeriodicSync() {
    val constraints = Constraints.Builder()
        .setRequiredNetworkType(NetworkType.CONNECTED)
        .build()
}

```

```

val syncWorkRequest = PeriodicWorkRequestBuilder<SyncWorker>(
    15, TimeUnit.MINUTES
)
    .setConstraints(constraints)
    .build()

WorkManager.getInstance(this).enqueueUniquePeriodicWork(
    SyncWorker.WORK_NAME,
    ExistingPeriodicWorkPolicy.KEEP,
    syncWorkRequest
)
}

```

NetworkConnectivityObserver.kt — отслеживание сети:

```

class NetworkConnectivityObserver(private val context: Context) {
    private val connectivityManager =
        context.getSystemService(Context.CONNECTIVITY_SERVICE)
            as ConnectivityManager

    fun observe(): Flow<NetworkStatus> {
        return callbackFlow {
            val callback = object : ConnectivityManager.NetworkCallback() {
                override fun onAvailable(network: Network) {
                    trySend(NetworkStatus.Available)
                }
                override fun onLost(network: Network) {
                    trySend(NetworkStatus.Lost)
                }
            }

            val request = NetworkRequest.Builder()
                .addCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET)
                .build()

            connectivityManager.registerNetworkCallback(request, callback)

```

```

        awaitClose {
            connectivityManager.unregisterNetworkCallback(callback)
        }
    }.distinctUntilChanged()
}
}

```

NotificationHelper.kt:

```

object NotificationHelper {
    const val CHANNEL_ID = "sync_channel"

    fun createNotificationChannel(context: Context) {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val channel = NotificationChannel(
                CHANNEL_ID,
                "Синхронизация",
                NotificationManager.IMPORTANCE_DEFAULT
            )
            val notificationManager = context.getSystemService(
                Context.NOTIFICATION_SERVICE
            ) as NotificationManager
            notificationManager.createNotificationChannel(channel)
        }
    }

    fun showSyncSuccessNotification(context: Context) {
        val builder = NotificationCompat.Builder(context, CHANNEL_ID)
        .setSmallIcon(R.drawable.ic_feed)
        .setContentTitle("Синхронизация")
        .setContentText("Новые данные получены")
        .setPriority(NotificationCompat.PRIORITY_DEFAULT)
        .setAutoCancel(true)

        NotificationManagerCompat.from(context)
    }
}

```

```
        .notify(1001, builder.build())
    }
}
```

item_message.xml — макет карточки сообщения:

```
<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:cardCornerRadius="12dp"
    app:cardElevation="4dp">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:padding="16dp">

        <com.google.android.material.imageview.ShapeableImageView
            android:id="@+id/image_avatar"
            android:layout_width="48dp"
            android:layout_height="48dp"
            app:shapeAppearanceOverlay="@style/CircleImageView" />

        <TextView android:id="@+id/text_message_name" />
        <TextView android:id="@+id/text_message_email" />
        <TextView android:id="@+id/text_message_body" />

        <ImageButton
            android:id="@+id/button_like"
            android:src="@drawable/ic_heart_outline" />

    </androidx.constraintlayout.widget.ConstraintLayout>
</com.google.android.material.card.MaterialCardView>
```



Рисунок 11 – страничка ленты с реализованными лайками

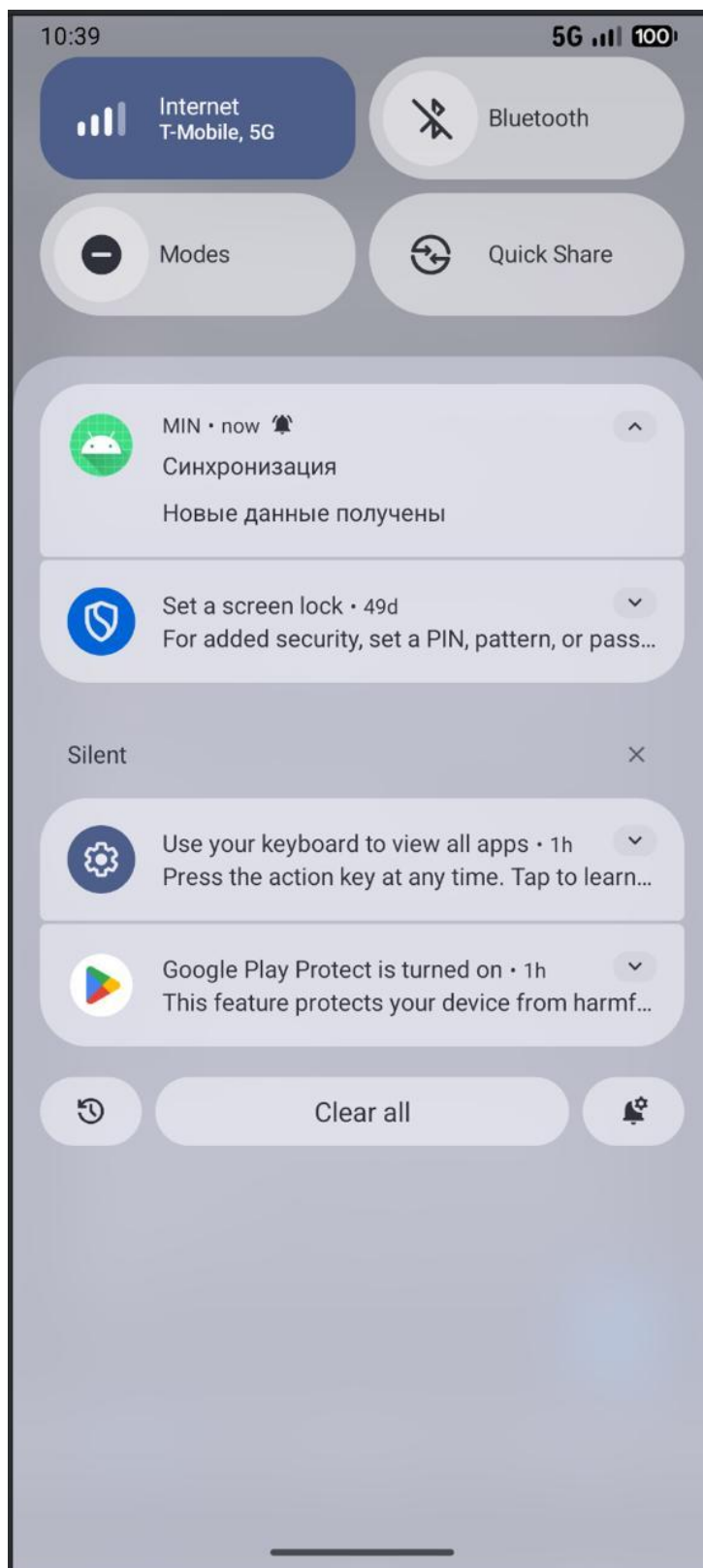


Рисунок 12 – демонстрация вывода уведомлений при успешной сборке данных

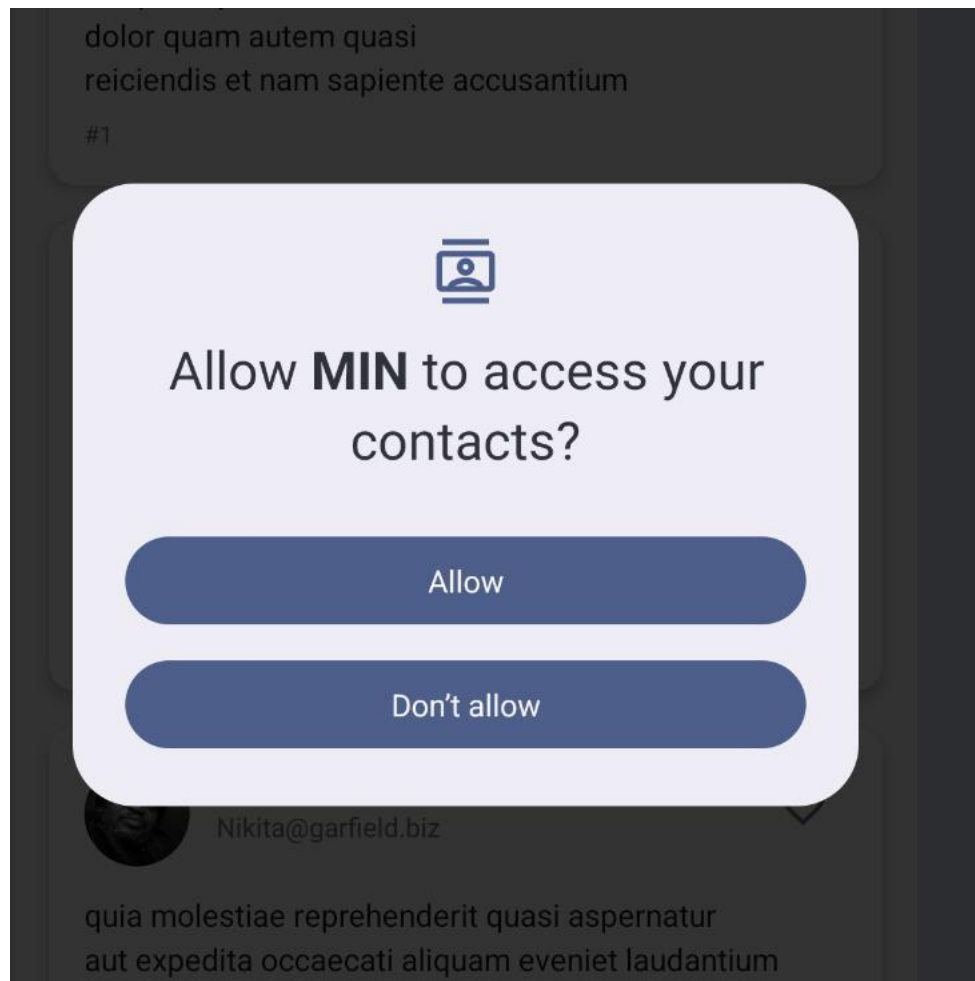


Рисунок 13 – запрос приложения при входе на доступ к данным на устройстве, в данном случае запрос доступа для контактов

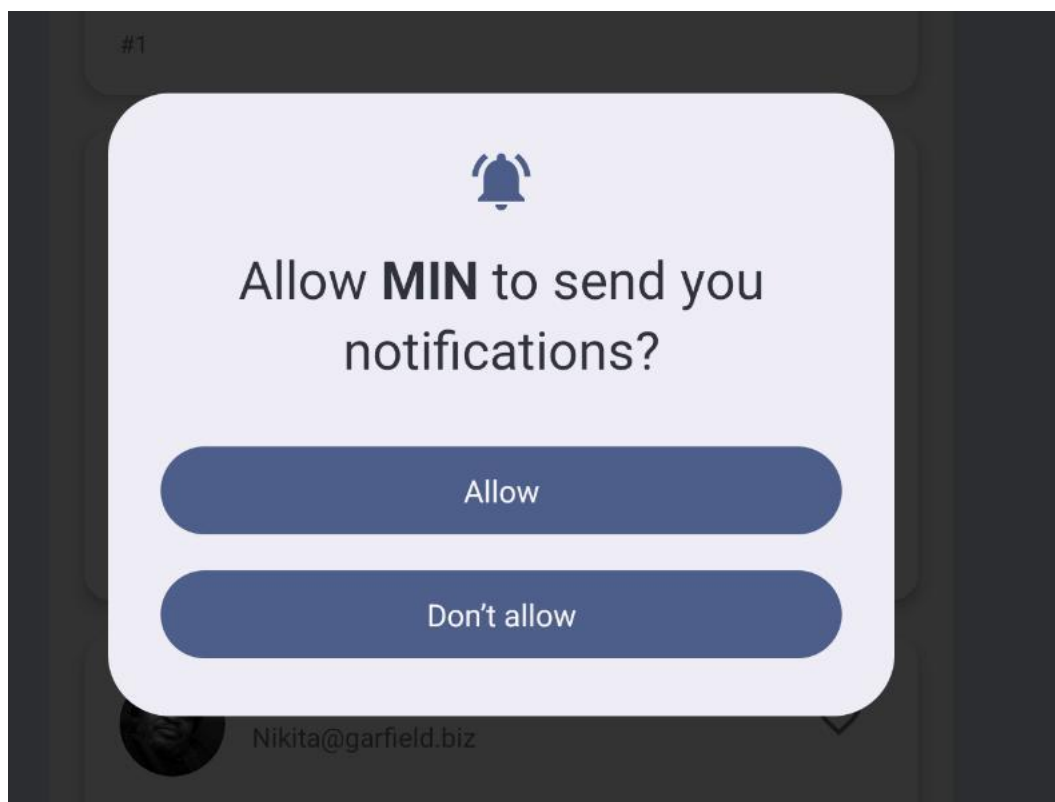


Рисунок 14 – запрос приложения при входе на доступ к данным на устройстве, в данном случае запрос доступа для отправлений уведомлений

6. Заключение

В ходе выполнения курсовой работы было разработано полноценное Android-приложение "MIN" — мессенджер, демонстрирующий применение современных технологий и архитектурных подходов.

Результаты работы:

Реализована навигация между экранами с использованием Navigation Component

Применена архитектура MVVM с разделением логики и представления

Настроена работа с сетью через Retrofit и кэширование в Room

Реализована фоновая синхронизация через WorkManager

Добавлены уведомления о синхронизации

Реализовано отслеживание состояния сети

Применены компоненты Material Design для современного интерфейса и также был разработан логотип для нашего приложения:



Рисунок 15 – логотип приложения

Вывод

Пройдя курс предмета мобильной разработки, мы научились делать базовые приложения для Android устройств на базе Kotlin, решая интересные кейсы на выходе мы получили приложение MIN, которое в дальнейшем можно доработать до полноценного мессенджера и начать конкурировать с его «старшим братом» - приложением MAX. Но если отбросить шутки, то курс был очень интересным и полезным.